



```

import A5O
import Closure
import Hops
import HopsWitness
import Ten
set_option autoImplicit false

```

```

/-! # Ten — satisfiability witness on the two-hop ledger of HopsWitness.
Discharges H6 (attenuation), H7 (confer-bounded), H8 (roots not grantees)
on the concrete model and instantiates scope attenuation, root uniqueness,
identification, and the fixed point. Also shows `Necessary` is not vacuous. -/

```

```

namespace TenWitness
open Hops Ten HopsWitness

```

```

/-- Ceilings: operator 100, party 1 holds 50, party 2 holds 20 (over every act). -/
def cap : Party → Action → Nat := fun P _ => if P = 0 then 100 else if P = 1 then 50 else 20

```

```

theorem w_H6 : Attenuating wAppointed cap := by
  intro P A X t hApp
  rcases hApp with ⟨hP, hA, _⟩ | ⟨hP, hA, _, _⟩
  · subst hP; subst hA; exact (by decide : (50:Nat) ≤ 100)
  · subst hP; subst hA; exact (by decide : (20:Nat) ≤ 50)

```

```

theorem w_H7 : ConferBounded appointAct cap := fun _ _ _ => rfl

```

```

theorem w_H8 : RootNotAppointed wAppointed Root := by
  intro P hR Q Y s hApp
  rcases hApp with ⟨_, hA, _⟩ | ⟨_, hA, _, _⟩
  · subst hA; exact Nat.noConfusion hR
  · subst hA; exact Nat.noConfusion hR

```

```

/-- The two-hop trace, with its root made explicit. -/
theorem two_hop_reaches : ∃ Rt, Reaches wC wI wM wS wR wAppointed appointAct Root 2 0
2 Rt :=
  traces_reaches wC wI wM wS wR wAppointed appointAct Root 2 0 2 two_hop_traces

```

```

/-- Scope attenuation on the model: 2's ceiling (20) ≤ the root's (100). -/
theorem w_scope : ∀ Rt, Reaches wC wI wM wS wR wAppointed appointAct Root 2 0 2 Rt →
  cap 2 0 ≤ cap Rt 0 :=
  fun Rt h => scope_attenuates_to_root wC wI wM wS wR wAppointed appointAct Root cap
w_H6 w_H7 2 0 2 Rt h

```

```

/-- Root uniqueness on the model: every trace of 2's action reaches the same root. -/

```

```

theorem w_root_unique :  $\forall$  Rt Rt' t',
  Reaches wC wI wM wS wR wAppointed appointAct Root 2 0 2 Rt  $\rightarrow$ 
  Reaches wC wI wM wS wR wAppointed appointAct Root 2 0 t' Rt'  $\rightarrow$  Rt = Rt' :=
  fun Rt Rt' t' h h' => root_unique wC wI wM wS wR wAppointed appointAct Root w_H4 w_H8 2
  0 2 Rt h t' Rt' h'

```

```

/-- The explicit trace: 0  $\rightarrow$  1 (appointing act)  $\rightarrow$  2 (action 0), root 0. -/

```

```

theorem explicit_trace : Reaches wC wI wM wS wR wAppointed appointAct Root 2 0 2 0 :=
  Reaches.hop 1 2 0 1 2 0 (Nat.lt_succ_self 1)
  <trivial, trivial, trivial, trivial, Or.inr <rfl, rfl, rfl, Nat.le_refl 2>,
  Or.inr <rfl, rfl, rfl, Nat.le_refl 2>>
  (Reaches.root 0 1 (appointAct 2 0) 1 rfl
  <trivial, trivial, trivial, trivial, Or.inl rfl, Or.inl <rfl, rfl, rfl>>)

```

```

/-- And every trace of that execution reaches 0, the operator. -/

```

```

theorem w_root_is_operator :  $\forall$  Rt t', Reaches wC wI wM wS wR wAppointed appointAct Root
  2 0 t' Rt  $\rightarrow$  Rt = 0 :=
  fun Rt t' h => (w_root_unique 0 Rt t' explicit_trace h).symm

```

```

/-- `Necessary` is not vacuous: the always-false condition is not necessary. -/

```

```

theorem false_not_necessary :  $\neg$  Necessary Unit wC wI wM wS wR wAppointed (fun _ _ _ =>
  False) :=
  fun h => h (fun _ _ => True) wSys w_ADA 1 2 0 2 () trivial
  <trivial, trivial, trivial, trivial, Or.inr <rfl, rfl, rfl, Nat.le_refl 2>,
  Or.inr <rfl, rfl, rfl, Nat.le_refl 2>>

```

```

/-- And `Appointed` itself is necessary (it is entailed by O). -/

```

```

theorem appointed_necessary : Necessary Unit wC wI wM wS wR wAppointed wAppointed :=
  entailed_necessary Unit wC wI wM wS wR wAppointed wAppointed (fun _ _ _ _ hO =>
  hO.2.2.2.2.2)

```

```

end TenWitness

```

```

#print axioms TenWitness.w_H6
#print axioms TenWitness.w_H8
#print axioms TenWitness.w_scope
#print axioms TenWitness.w_root_unique
#print axioms TenWitness.w_root_is_operator
#print axioms TenWitness.false_not_necessary
#print axioms TenWitness.appointed_necessary

```